

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

Sorting III

COUNTING SORT

Assume every element is an integer in the range 0 to k-1. Then the key is already an array index.

INPUT	INDEX	COUNT ARRAY
[4, 1, 0, 4, 3, 1]	[0, 1, 2, 3, 4]	[1, 2, 0, 1, 2]
PASS	ARRAY SO FAR	
1	[0]	
2	[0, 1, 1]	
3	[0, 1, 1]	
4	[0, 1, 1, 3]	
5	[0, 1, 1, 4, 4]	

COUNTING SORT

Assume every element is an integer in the range 0 to $k-1$. Then the key is already an array index.

```
def counting_sort(arr, k):  
    counts = [0] * k  
    for x in arr:  
        counts[x] += 1  
  
    for d in range(1, k):  
        counts[d] += counts[d - 1]  
  
    out = [0] * len(arr)  
    for x in reversed(arr):  
        counts[x] -= 1  
        out[counts[x]] = x  
    return out
```

THREE PASSES, ALL LINEAR

Count each value: $\Theta(n)$. Turn counts into running totals: $\Theta(k)$. Place each element at its computed index: $\Theta(n)$. Total $\Theta(n + k)$, with no comparison anywhere.

WHY THE LAST LOOP RUNS BACKWARD

The running total is the index just past the last slot for that value. Walking the input in reverse and decrementing hands out slots back to front, so equal keys keep their original order. Counting sort is stable, and the next algorithm depends on it.

RADIX SORT RUNS COUNTING SORT ONCE PER DIGIT

One pass per digit, least significant first, each pass a stable counting sort on that digit alone. Nothing is ever compared.

PASS	SORT KEY	ARRAY AFTER THE PASS
start	unsorted	170 045 075 090 802 024 002 066
1	ones digit	170 090 802 002 024 045 075 066
2	tens digit	802 002 024 045 066 170 075 090
3	hundreds digit	002 024 045 066 075 090 170 802

LINEAR, WITH CONDITIONS ATTACHED

$$T(n) = \Theta(d \cdot (n + k)) \quad d = \text{number of digits}, k = \text{size of the digit alphabet}$$

WHEN IT WINS

Fixed width integer keys, a small base, and d that does not grow with n . Then d and k are constants and the total is $\Theta(n)$.

Sorting n 32-bit integers in base 256 is 4 passes, no matter how large n gets.

WHEN IT LOSES

Keys that are not integers, keys whose width grows with n , or a base large enough that k dominates.

It also allocates $\Theta(n + k)$ per pass and is never in place, so a memory-bound workload can lose on the constants alone.

TIMSORT. WHAT PYTHON ACTUALLY RUNS.

REAL DATA IS ALREADY PARTLY SORTED

`sorted()` and `list.sort()` in Python, and `Arrays.sort()` on objects in Java, all run Powersort (a variation of Timsort). It is a merge sort that looks at the input first.

01 Find the runs

Scan left to right for maximal stretches that are already ascending, or strictly descending. A descending run is reversed in place, which is why the reversal test must be strict: reversing a stretch of equal keys would break stability.

02 Extend short runs to minrun

minrun is chosen between 32 and 64 so that the number of runs is close to a power of two. A run shorter than minrun is grown with binary insertion sort, which is $\Theta(n^2)$ but on a block of at most 64 elements with excellent locality.

03 Merge runs, never blindly

Runs are pushed on a stack and merged in an order that keeps their lengths balanced. Merging is the same procedure from last lecture, and it is where the $\Theta(n)$ extra space goes.

THE PARTS THAT ARE NOT OBVIOUS

THE STACK INVARIANT For the top three run lengths X, Y, Z on the stack, Timsort maintains $Z > Y + X$ and $Y > X$, merging whenever that fails. The invariant forces merges to happen between runs of similar length, which is what keeps the total at $\Theta(n \log n)$.	GALLOPING MODE If one run wins several consecutive comparisons, the merge switches to binary search for the insertion point of the next element. When one run is entirely smaller than the other, this turns $\Theta(n)$ comparisons into $\Theta(\log n)$.	THE 2015 BUG The Java port under-enforced the invariant. A constructed input could overflow the fixed size run stack and raise an exception. It was found by formally verifying the implementation, not by testing it.		
BEST CASE	WORST CASE	STABLE	EXTRA SPACE	ADAPTIVE
$\Theta(n)$	$\Theta(n \log n)$	Yes	$\Theta(n)$	Yes

This is the algorithm the book points at when it says correctness can be hard to prove. It is also the one running behind every `sorted()` call you have written. Both facts are worth holding at once.

CLOSING THOUGHTS